Atentiv Machine Learning & Dataset Specification

Complete Technical Guide: Training Data Sources, Preprocessing Parity & fastText WASM Inference

Target Architecture: Chromium MV3 Extension (WASM) + Python 3.11

Core ML Model: fastText Supervised Quantized (`.ftz`)

Taxonomy: 13 Canonical Productivity & Topic Classes

Privacy Guarantee: 100% Local-First / Zero Cloud Network Egress

1. Executive Summary & Problem Formulation

Modern web analytics tools rely on cloud telemetry, transmitting user URLs, window titles, and page DOM extracts to remote machine learning APIs. This compromises confidentiality and violates corporate privacy compliance. **Atentiv** resolves this by executing full on-device natural language classification locally in the browser runtime via WebAssembly (fastText WASM), with **zero bytes transmitted over the network**.

To achieve high categorization accuracy with zero cloud latency, Atentiv requires an offline supervised model trained on diverse web domain distributions and a real-time feature extractor that matches the training text *character-for-character*.

2. Which Data is Used for Training? (Dataset Composition)

The training dataset was engineered from a mixture of open-source curated web corpora, web navigation logs, and synthetic edge-case augmentations:

Dataset Source	Composition & Domain Coverage	Volume / Samples	Taxonomy Target
Common Crawl Web Subset	Pre-scraped HTML titles, meta-descriptions, and body snippets from top 50,000 domains.	~85,000 pages	News, Reference, Shopping, Government, Finance
Mind2Web / WebUI Corpus	DOM hierarchical text, interactive UI controls, and documentation web pages.	~42,000 pages	Development, Productivity, Reference
Curated Domain Seed DB	Hand-labeled ground truth for top technical, corporate, and productivity tools (GitHub, StackOverflow, Jira, Figma, Gmail, Notion).	~12,500 pages	Development, Productivity, Communication
Media & Streaming Metadata	YouTube video titles/channels, Spotify podcasts/playlists, Twitch livestreams, and Soundcloud tracks.	~18,000 streams	Education, Entertainment, Communication
Adversarial & Edge Cases	Synthesized short titles, single-token pages, 404 error pages, login portals, and noisy URLs.	~5,000 samples	System, General, Untracked

> **Key Quality Rule: Domain-Stratified Splitting (80/10/10)**
> To prevent data leakage, splitting was performed at the *domain level* rather than the page level. If `github.com` is in the training set, all GitHub pages are strictly confined to training, ensuring the validation and test sets measure genuine generalization on unseen domains.

3. The 13 Canonical Taxonomy Classes

The model classifies text into 13 standardized productivity and topic categories:

1. **Development:** Code repositories, developer tools, API documentation, terminal logs.
2. **Productivity:** Task trackers, project management, word processors, spreadsheets (Jira, Notion, Docs).
3. **Communication:** Email, workplace chat, video conferencing (Slack, Teams, Meet).
4. **Education:** University portals, computer science courses, instructional talks, research papers.
5. **Reference:** Encyclopedias, technical specifications, documentation hubs (Wikipedia, MDN, StackOverflow).
6. **Entertainment:** Video streaming, social entertainment, gaming streams, sports highlights.
7. **Social Media:** Personal social networks, microblogging, community forums (Reddit, X/Twitter, Instagram).
8. **News & Media:** Journalism, broadcast press, editorial magazines, current event reports.
9. **Shopping:** E-commerce stores, retail marketplaces, product price aggregators.
10. **Finance:** Banking portals, cryptocurrency exchanges, stock trading, accounting tools.
11. **Health & Fitness:** Medical reference, wellness portals, workout tracking.
12. **Government:** Public sector portals, civic services, tax and licensing websites.
13. **General / Untracked:** New tab pages, settings, blank documents, unclassified web utilities.

4. How the Model is Made: Preprocessing & Training Pipeline

The model training and deployment pipeline involves 6 sequential phases designed to guarantee cross-language parity:

Phase 1: Feature Extraction (DOM Context Synthesis)

In the browser, the content script extracts three lightweight text streams:

- **URL Path & Host:** Stripped of query credentials, tokens, session IDs, and tracking parameters.
- **Page Title:** Document title tag, trimmed and cleaned.
- **Meta Description & Headings:** Standardized extract of up to 20 heading tags (H1-H3) and the meta description (capped at 2 KB to prevent latency spikes).

Phase 2: Text Normalization (The Parity Contract)

Because training occurs in **Python** but real-time inference executes in **TypeScript via WebAssembly**, both pipelines must normalize text with identical mathematical rules:

1. **Unicode Normalization:** Full Unicode NFKC normalization to resolve composite glyphs and ligatures.
2. **Case Folding:** Complete lowercase conversion.
3. **URL & Token Cleaning:** Stripping tracking parameters (`utm_*`, `fbclid`, `gclid`), session cookies, and numeric ID fragments.
4. **Whitespace Collapsing:** Converting tabs, carriage returns, and multiple spaces into a single space.
5. **Token Truncation:** Text payload capped at the first 256 tokens to guarantee inference executes under 10 milliseconds.

```
Python Pipeline: Unicode NFKC -> lowercase -> regex sanitize -> whitespace collapse -> 256 tokens TypeScript WASM: normalize("NFKC") -> toLowerCase() -> regex sanitize -> trim() -> 256 tokens Parity Assertion: 200/200 Golden Fixtures match 100% character-for-character.
```

Phase 3: Supervised fastText Training & Hyperparameters

The model is trained using Facebook AI's `fastText` supervised learning algorithm. fastText represents words as bags of character n-grams, enabling robust classification even for unseen typos or obfuscated URLs.

Hyperparameter	Setting	Architectural Justification
Algorithm	Supervised FastText	Linear classifier with rank constraint, highly parallelizable.
Learning Rate (lr)	0.25	Fast convergence over large vocabulary corpus.
Embedding Dimension (dim)	64	Compact vector size; keeps model footprint under 4 MB.
Epochs	25	Prevents overfitting while optimizing multi-class cross-entropy.
Word N-Grams	2 (Bigrams)	Captures crucial compound phrases (e.g., "deep learning", "lo fi beats").

Hyperparameter	Setting	Architectural Justification
Loss Function	Hierarchical Softmax (hs)	O(log K) evaluation complexity, essential for instant browser responsiveness.

Phase 4: Quantization & WASM Compilation

The raw trained model (`.bin`, ~48 MB) is quantized via vector quantization:

- Pruning low-frequency n-grams and vocabulary embeddings.
- Compressing 32-bit floating point weights into 8-bit quantized integer tables.
- Outputting `atentiv-classifier.ftz` (**3.58 MB**), small enough to bundle directly inside the extension zip.
- Generating an SHA-256 integrity hash in `model_manifest.json` to ensure tamper-proof loading in Chrome MV3.

5. Evaluation & Parity Verification

The model was tested against both offline held-out test splits and cross-language parity fixtures:

- **Cross-Language Parity (TS vs. Python):** 200 Golden Fixture URLs and titles were normalized in both Python and TypeScript. **Result: 200/200 matched 100% character-for-character (0 discrepancy).**
- **Inference Latency:** Averaging **8.9 ms** per page classification on client CPU.
- **Memory Footprint:** WebAssembly heap allocation strictly bounded under **16 MB**.
- **Telemetry Invariant:** 0 outbound HTTP/HTTPS requests generated during classification.